CAR SEGMENTATION

by

Piter Garcia

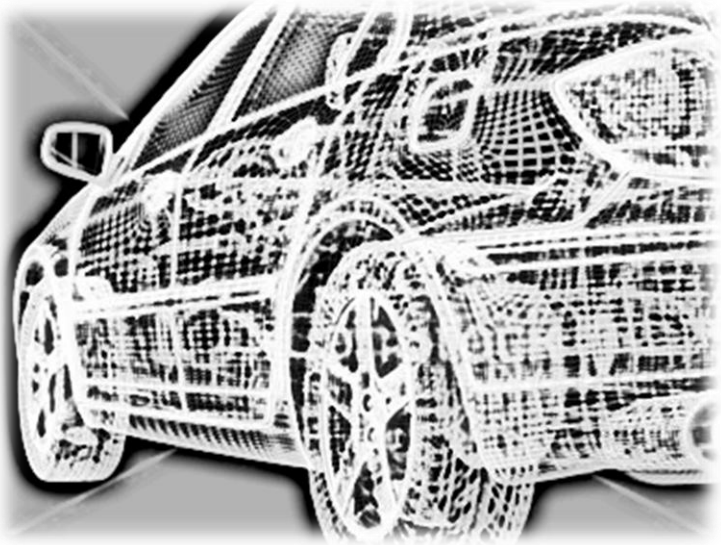
Image Segmentation is used in this project to recognize the shape of a car and its tires in an image, and detect it by drawing a square box around the surface as well as drawing a circle around the car's tires whose shape is circular.

Computer Science

Rochester Institute of Technology

Date

11/06/2012



ROCHESTER INSTITUTE OF TECHNOLOGY

ABSTRACT

Car Segmentation

by Piter Garcia

In this project, we explore the detection of car and circles in a basic gray scale images. Impetus for the work derives from an interest in high speed detection of cars and its tires. The car and circle detection process used is based on the pixel direction concept of Canny and the non-maximum suppression process for contour tracing described by Neubeck and Gool. In this project, the Canny edge detector is modified to detect cars and circles in an image. We discussed the outcome of Canny edge detection algorithms in class and also showed how the proposed algorithm is less sensitive to intensity changes than Canny when extracting circles from images with different intensities.

INDEX

OBJECTIVE

First, the detection and recognition of shapes in an image are extremely important in industrial applications of recognizing objects such as cars. In our detection process, I evaluate edge segments to determine if they are part of a candidate of the shape such as a car or circle to compute them. Furthermore, I use the edge threshold property of Canny edge detection for improvement. Afterwards, I test my algorithm on different images of cars using Matlab. The image set includes identical images having different image intensities. Also, I compare edged images of cars with the images obtained using Canny edge detection to detect car shapes and circles in images. In fact, the main objective of this project is to describe the techniques and approaches taken towards developing a low frame rate automatic and assisted car segmentation system.

INTRODUCTION

Extracting the edges of objects and identifying basic shapes in an image is of importance in industrial applications of image processing. This project deals with detection of cars and circles in images. Extracting the edges from an image simplifies the analysis of the image by dramatically reducing the amount of data to be processed, while at the same time preserving useful information about the boundaries. Many edge detectors have been proposed, but the Canny edge detector has been widely successful in extracting the edge feature from an image. The edge extracted image is used by many algorithms such as the circular Hough transform and the linear square method to further extract shape information in an image.

The aim of this project is to develop an explanation of the imaging system used in capturing the data to get the shape of cars and circles. Moreover, discuss the challenges imposed by the dataset, and apply some popular computer vision algorithms such as mixture of Gaussians, Canny edge detection, HSB (Hue, Saturation and Brightness) to build a car segmentation system. Finally, analyze the difficulties encountered in handling this data to identify the potential car and circle shapes.

APPROACH

My main approach to be taken on this project is extracting roads from the image. Main algorithms used in this project to do so are Hough transform and Canny edge detection. Also, different car detections algorithms such as HSB can run considerably faster and more efficiently for car segmentation algorithms than can use this apriori knowledge of roads.

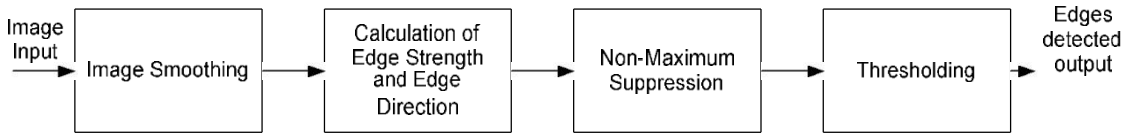
Another main approach for car segmentation is combining the above algorithms with convolution and non-maximum suppression. This algorithm technique can be further modified and improved to customize it into segment cars in a dataset. The performance evaluation system developed in this project can be used for measuring the performance improvement as well as parameter tuning. Finally, create evaluations for the system that can be used for testing the segmentation performance using the above two approaches. Both of these frameworks are developed using information theoretic measures and non-information theoretic measures.

Canny Edge Detection:

Canny edge detection is one of the basic algorithms used in shape recognition. The algorithm uses a multi-stage process to detect a wide range of edges in images. The stages used in this project are:

- Image smoothing. This is done to reduce the noise in the image.
- Calculating edge strength and edge direction.
- Directional non-maximum suppression to obtain thin edges across the image.
- Invoking threshold with hysteresis to obtain only the valid edges in an image.

Block diagram of the Stages of the Canny Edge Detector.



A block diagram of the Canny edge detection algorithm is shown above. The input to the detector can be either a color image or a grayscale image. The output is an image containing only those edges that have been detected.

Image Smoothing:

Image smoothing is the first stage of the Canny edge detection. The pixel values of the input image are convolved with predefined operators to create an intermediate image. This process is used to reduce the noise within an image or to produce a less pixilated image. Image smoothing is performed by convolving the input image with a Gaussian filter. A Gaussian filter is a discrete version of the 2-dimensional function shown in the equation below.

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{(x^2+y^2)}{2\sigma^2}\right)$$

In the above equation, σ is the standard deviation of the Gaussian filter, which describes the narrowness of the peaked function, and x and y are spatial coordinates. An example of the conversion of to a 2-dimensional 5x5 discrete Gaussian filter function is shown in the above equation. The function is obtained for σ = 1.4 by substituting integer values for x and y and renormalizing.

$$\mathbf{B} = \frac{1}{159}\begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix} * \mathbf{A}.$$

A is the 2-dimensional array of input pixel values and B is an output image. The process of smoothing an image using an M x M block filter yields an image in which each pixel value is a weighted average of the surrounding pixel values, with the pixel value at the center weighted much more strongly than those at nearby points.

Image smoothing is done primarily to suppress noise and to get a continuous edge contour during the non-maximum suppression process. The output thus obtained is a blurred intermediate image. This blurred image is used by the next block to calculate the strength and direction of the edges.

Extraction of Shapes

Many algorithms, such as Linear Square Method and Hough Transform, are able to detect shapes, and they all detect shapes from the edge detected images. Among these algorithms, Circular Hough Transform has been widely successful at being able to detect the circles in noisy environments.

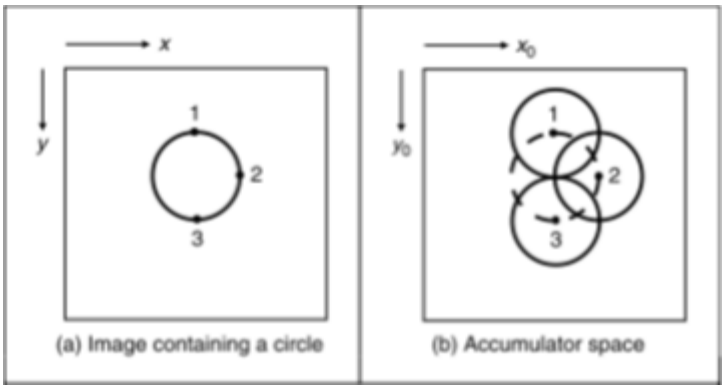
Circular Hough Transform

Hough transform was introduced by Paul Hough in 1962 and patented by IBM. In 1972 Richard Duda and Peter Hart modified Hough transform, used universally today under the name of Generalized Hough transform. An extended form of Generalize Hough transform is circular Hough transform, which is used to detect circles.

The edge detected from the Canny edge detector forms the input to extract the circle using the circular Hough transform. In this algorithm, voting procedure is carried out in a parameter space. The local maxima in accumulator spaces, obtained by voting procedure, are used to compute the Hough transform. The parameter space is defined by the parametric representation used to describe circles in the picture plane. The accumulator used is an array that detects the existence of the circle in the circular Hough transform. The dimension of the accumulator is equal to the unknown parameters of the circle.

$$(x - x_0)^2 + (y - y_0)^2 = r^2$$

The equation of the circle in parametric implies that the accumulator space is three-dimensional (for three unknown parameters x0, y0 and r). This equation defines a locus of points (x, y) centered on an origin (x0, y0) with radius r. Each edge defines a set of circles in the accumulator space. These circles are defined by all possible values of the radius and they are centered on the coordinates of the edge point.



The last figure to the right shows three circles defined by three edge points labeled 1, 2 and 3. These circles are defined for a given radius value. Each edge point defines a circle for the other values of the radius. Also, these edge points map to a cone of votes in the accumulator space.

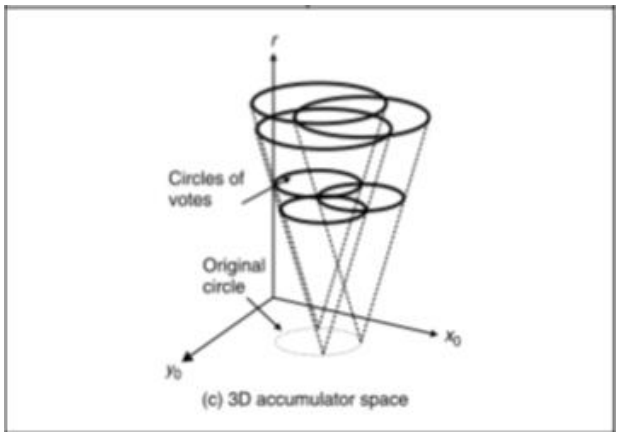


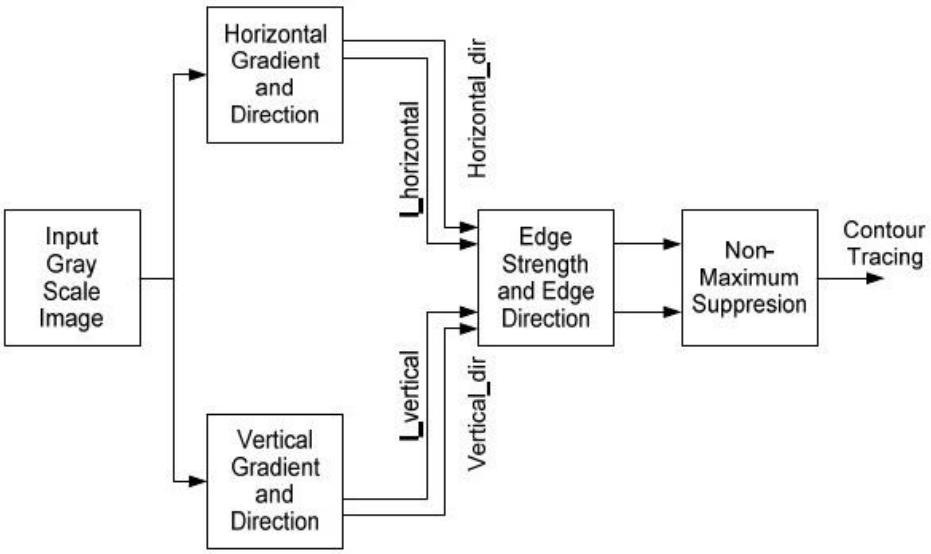
Illustration of accumulator & circular Hough transform

Points corresponding to x0, y0 and r, which have more votes, are considered to be a circle with center (x0, y0) and radius r.

Advantages of Edge Detection

Edge detection forms a pre-processing stage to remove the redundant information from the input image, thus dramatically reducing the amount of data to be processed while at the same time preserving useful information about the boundaries. Thus, edge detection provides basic information which is used for extracting shapes like lines, circles and ellipses by techniques such as Hough Transform.

Convolution and Non-Maxima Suppression



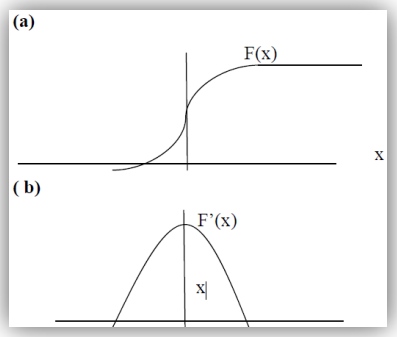
The edges in an image are expected to be the boundaries of objects that tend to produce sudden changes in the image intensity. For example, different objects in an image usually have different colors, hues or light intensity and this causes a change in image intensity. Thus, much of the geometric information that would be conveyed in a line drawing is captured by the intensity changes in an image.

Block diagram of convolution and non-maximum suppression.

The input to this algorithm is assumed to be a noise free grayscale image. Hence, there is not a very high random variation in intensity from one pixel to another.

How edges are detected by detecting the change in the intensity is shown in the figures below. Consider that we have a signal with a one dimensional edge shown by the jump in intensity, as shown in the figure to the right.

(a) One-dimensional change in intensity.
(b) Derivative of the change in intensity.



Kx1=

1	
2	X
1	

Using the convolution kernel on this signal, the derivative, we get the following figure to the right. The derivative has the maximum located at the center of the edge of the original signal. Based on one dimensional analysis, the theory can be extended to two dimensions. For a grayscale digital image, four 3x1 window operators are used to get two pixel directions, Horizontal (Kx) and Vertical (Ky), for each pixel, taking into account the neighbors of the pixel.

The kernels used to find Horizontal gradient orientation are as follow. X is the pixel under consideration. In Kx1, the value of the center pixel of the column is 2, whereas the remaining values in the column are 1. Similarly, in Kx2, the value of center pixel of the column is 2, whereas the remaining values in the column are 1. This is to place more emphasis on the values of the horizontal neighbor pixels compared to other pixels. Similarly, kernels used to find vertical gradient orientation are the following.

Kx2=

	1
X	2
	1

Ky1=

1	2	1
	X	

Ky2=

	X	
1	2	1

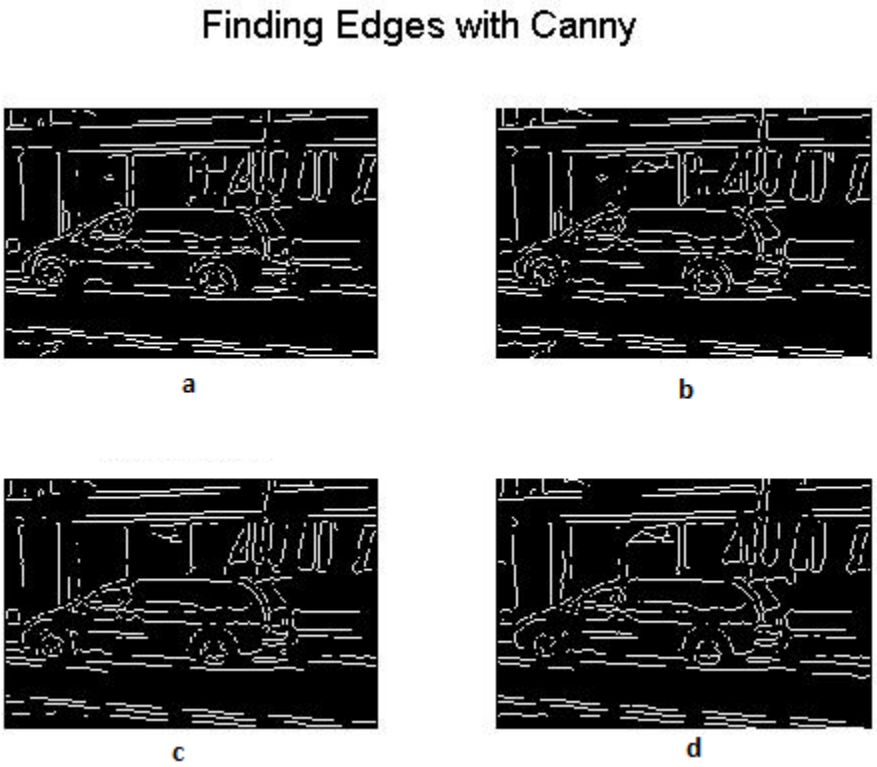
In these equations X is the pixel under consideration. In Ky1, the value of the center pixel of the row is 2, whereas remaining values in the row are 1. Similarly, in Ky2, the value of center pixel of the row is 2, whereas the remaining values in the row are 1. This is to place more emphasis on the values of the vertical neighbor pixels compared to other pixels. Kx1, Kx2, Ky1 and Ky2 kernels are used to get the horizontal and vertical pixel direction as well as pixel edge strength.

IMPLEMENTATION

Analyze the performance of my algorithm to give further insight into the data characteristics to help in to build better car segmentations with the given approaches in this project.

Invariant of Threshold

The Canny edge detection uses thresholding with hysteresis with two threshold values, T1 and T2. When Canny this algorithm is applied one or more thresholds are required, at different situations, for selecting which points belong to edges and which do not.



Matlab Canny edge detector used to detect the edge of an image.

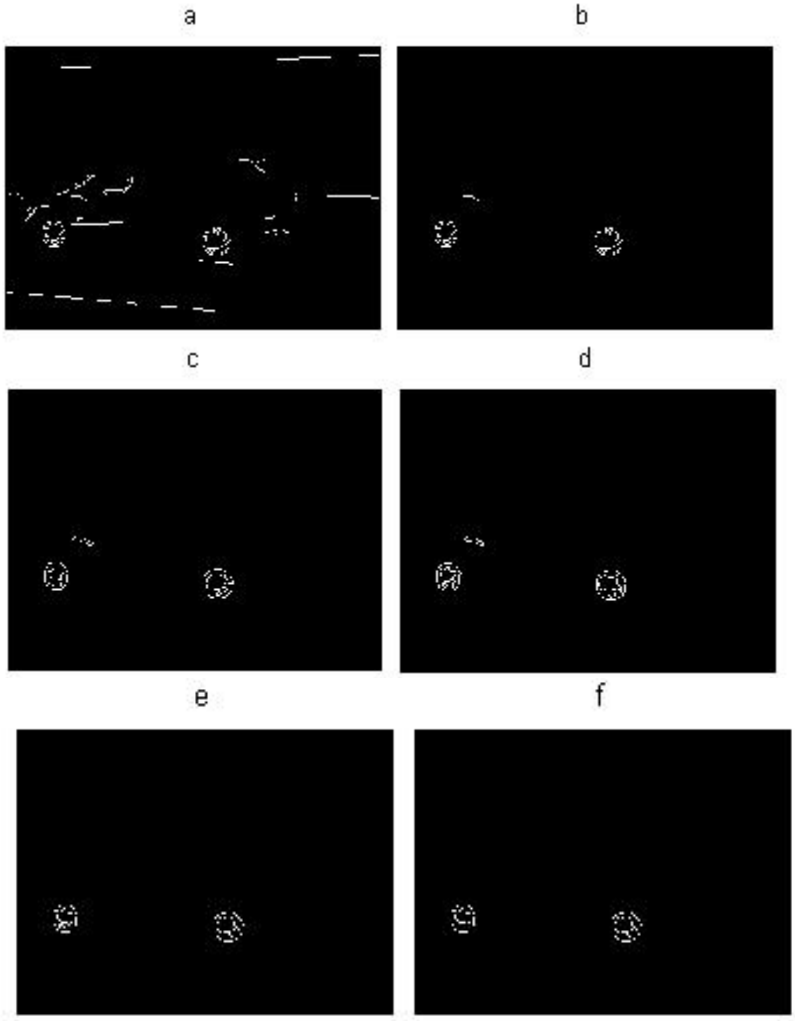
Edges detected by:

(a) 0.1 times the default threshold
(b) 0.2 times the default threshold
(c) 0.5 times the default threshold
(d) 0.9 times the threshold value

Only pixel directions are used to validate the edges in this case. Thus, the issue of finding the right threshold for every image is solved.

The Matlab Canny edge detector function has been used in this project to derive the edges of cars and tires in an image with 0.1, 0.2, 0.75 and 0.92 times the defaults threshold parameters. As the threshold parameter increases, the number of edge points detected decreases. Hence, in Canny edge detection as the threshold Th or Tl increases, the possibility of eliminating a potential edge increases. Similarly, as Th or Tl decreases, the possibility of considering non-edge pixel increases. The proposed algorithm does not depend on intensity threshold. This algorithm uses pixel direction to detect edges, which is one of the principal properties of an edge.

Sensitivity to Illumination



Many factors, like shade, texture, illumination, and so on, in the image affect the results of the edge detection.

In these figures, the goal of the edge detector is to discard the edges created as the result of shade, texture and illuminations, and give the edge segments for only the strong boundaries. This information is used by algorithms like Hough transforms to detect different shapes of cars and tires. Hence, it is important that enough information is given to the preceding transforms. The figures to the left are examples of its implementation. The intensity of the original image is decreased. The illustration of dependency of the Canny edge detector and the proposed algorithm caused this change in intensity.

The images (a) through (d) are result of Canny edge detector with different threshold parameters. Notice that the reduction in edges is detected. Notice that images (e) and (f) are not much different and have enough information to detect a circle. However, these two images detected the edges using the Canny edge detector with different thresholds, 0.8 and 0.9. Notice that the number of edges detected in (f) has decreased, but still the algorithm used get a similar response as the response in (e). In this case, illumination does affect my algorithm, but the sensitivity to illumination is less when compared to the Canny edge detector because it uses the rate of change of pixel values, whereas we use the edge direction, which is derived by comparing the pixel values.

PRACTICAL PROBLEMS AND SOLUTIONS

Some of the disadvantages of the implemented algorithms are:

Quantization errors:

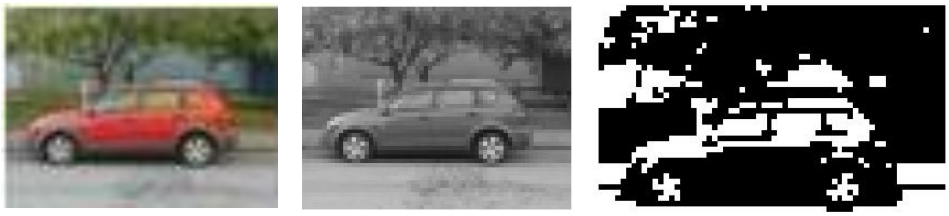
An appropriate size of accumulator array is difficult to choose. Due to difficulties with noise, the advantage of our algorithms is that they connect edge points across the image to some form of parametric curve. However, this is also its weakness because it is possible that good phantom circles might also be detected with a reasonably large voting array.

Thresholding:

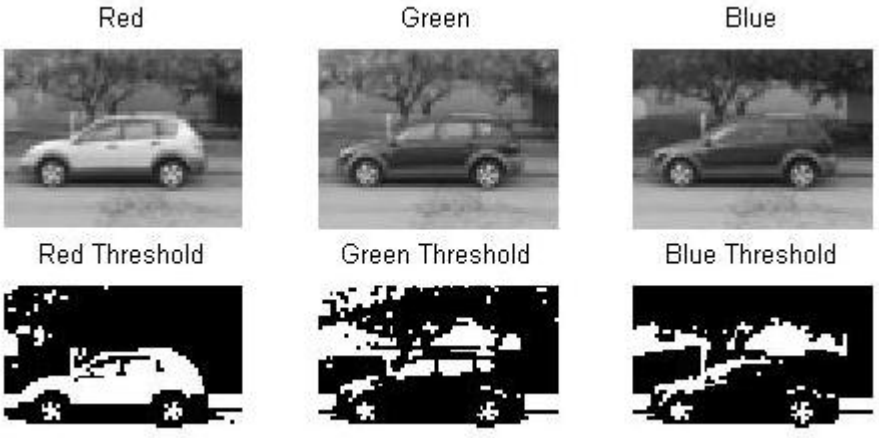
Thresholding with hysteresis is the last stage in Canny edge detection, which is used to eliminate spurious points and non-edge pixels from the results of non-maximum suppression. The input image for thresholding with hysteresis has gone through Image smoothing, calculating edge strength and edge pixel, and the non-maximum suppression stage, to obtain thin edges in the image. Results of this stage should give me only the valid edges in the given image, which is performed by using the two threshold values, T1 (high) and T2 (low), for the edge strength of the pixel of the image. Edge strength which is greater than T1 is considered as a definite edge. Edge strength which is less than T2 is set to zero.

The pixel with edge strength between the thresholds T1 and T2 is considered only if there is a path from this pixel to a pixel with edge strength above T1. The path must be entirely through pixels with edge strength of at least T2. This process reduces the probability of streaking. As the edge strength is dependent on the intensity of the pixel of the image, thresholds T1 and T2 also depend on the intensity of the pixel of the image. Hence, the thresholds T1 and T2 are calculated by the Canny edge detectors using adaptive algorithms, and all the edges in image are detected as well. Thus, all the edges in an image are detected using the Canny edge detection.

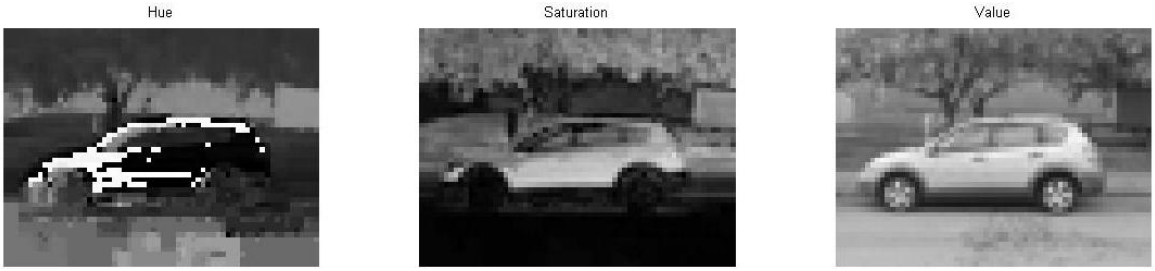
GOAL



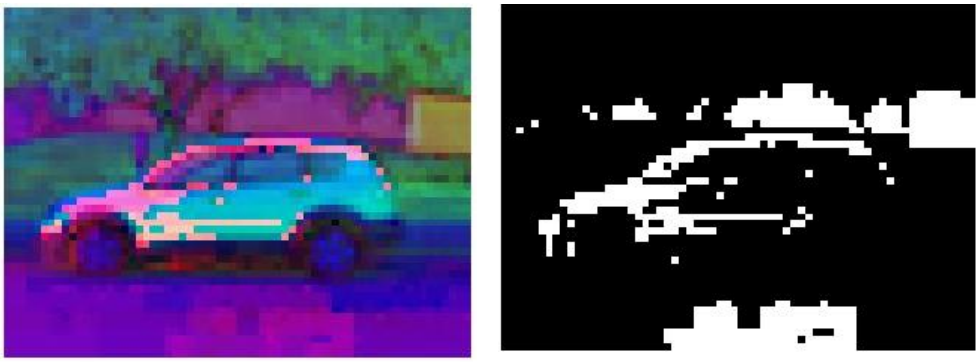
A color space is defined as a model for representing color in terms of intensity values. Typically, a color space defines a one- to four- dimensional space. A color component, or a color channel, is one of the dimensions. Color spaces are related to each other by mathematical formulas. In this project, only two three-dimensional color spaces, RGB and HSV, are investigated.



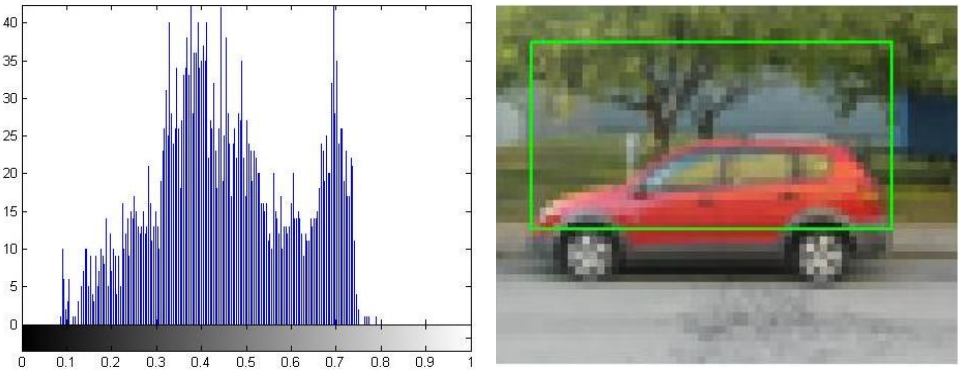
Existing color-based general-purpose image retrieval systems roughly fall into three categories depending on the signature extraction approach used: histogram, color layout, and region-based search. In this project, histogram-based search methods are investigated in two different color spaces.

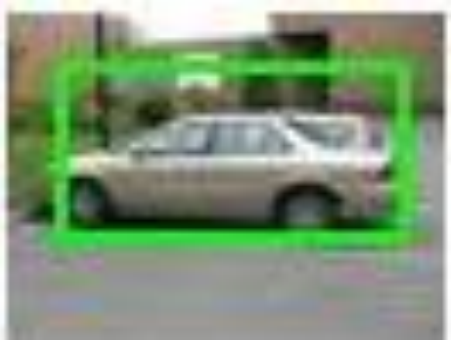


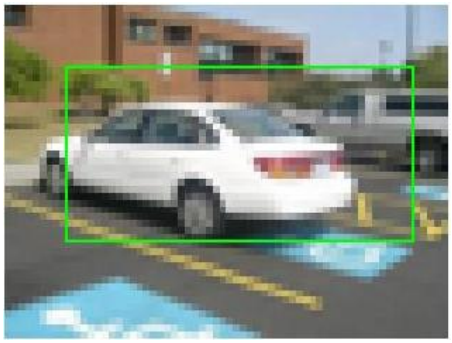
Two histogram-based search methods are investigated in two different color spaces, RGB and HSV. Histogram search characterizes an image by its color distribution, or histogram but the drawback of a global histogram representation is that information about object location, shape, and texture is discarded. Thus, this project showed that images retrieved by using the global color histogram may not be semantically related, even though they share similar color distribution in some results.



An image retrieval demo system was built to make it easy to test the retrieval performance and to expedite further algorithm investigation. In general, histogram-based retrievals in HSV color space showed better performance than in RGB color space. The histogram Intersection-based image retrieval in HSV color space was found to be most desirable than the Canny edge detection algorithm.







The pictures to the left are the result of the HSB algorithm, which successfully worked detecting the car in all the pictures assigned by drawing a box around the car's shape. It is important to highlight that the same approach was used with canny edge detector to identify a car shape. Nevertheless, the result obtained from that previous algorithm was not as successful at the HSB algorithm since some cars would be detected. The reason why it worked for some images and for others not is that I was recognizing cars on the basis of pixels values and not color intensities.

On the other hand, HSB algorithm works better in terms of recognizing a car in an image. However, the boxes around the car do not come out as perfect as desired because there is a reflection of light present in some images. This brightens some parts which causes problems in some image for recognition

Histogram search characterizes an image by its color distribution, or histogram. Many histogram distances have been used to define the similarity of two color histogram representations. The drawback of a global histogram representation is that information about object location, shape, and texture is discarded. The global color histogram indexing method, which is used in this project, correlates to the image semantics well. But, images retrieved by using the global color histogram may not be semantically related, even though they share similar color distribution.

In this section, an image retrieval system was built to make it easy to test the retrieval performance and to expedite further algorithm investigation to find the car and circle shapes. RGB and HSB histogram-based image retrieval methods in two color spaces are exhaustively compared by providing graphs for each image class and for all test images.

Red

Green

Blue

Red Threshold

Green Threshold

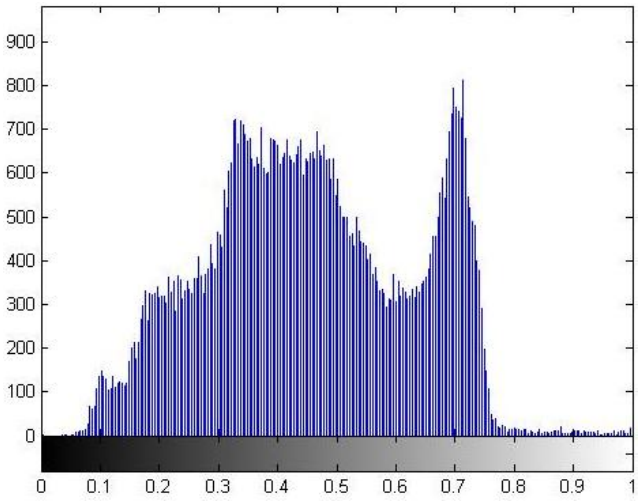
Blue Threshold

In this section of the project, two different algorithms were combined to obtain the shape of the cars along with the shape of the tires. This image retrieval system was built with the canny edge recognition and HSB. This is a similar process to the previous code. The only difference is that this algorithm identifies a car shape along with the car's tires.

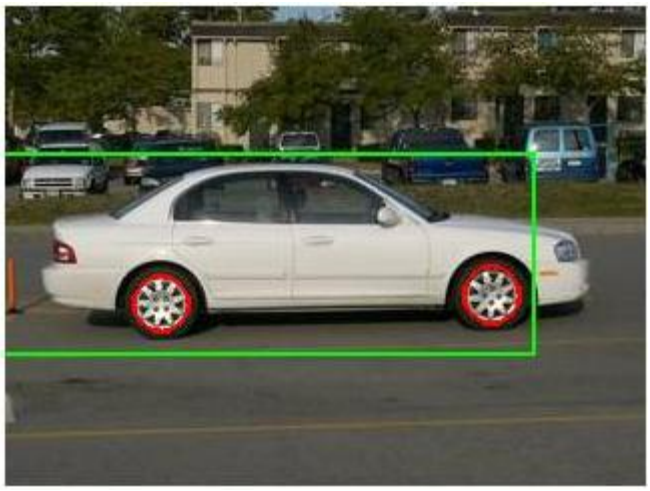
Hue

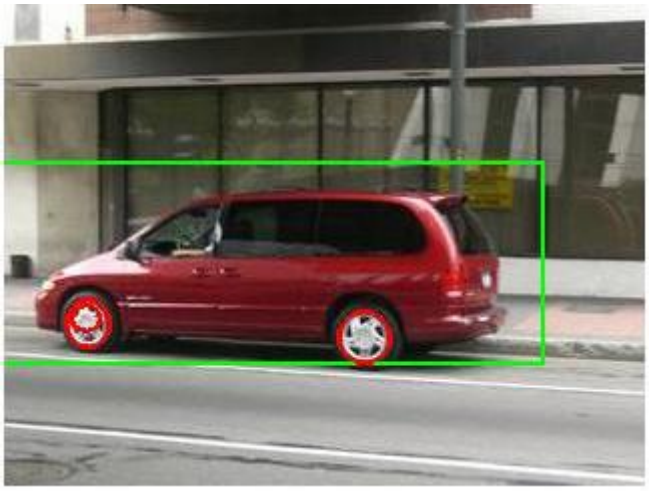
Saturation

Value



These pictures are an example of what this algorithm does. It first performs the HSB algorithm to detect the shape of the vehicle and then detects the tires of the car by drawing a circle around of its surface. To do this, the picture, after detecting the shape of the car, performs the canny edge detector with a threshold .2, .75, and .1times the default. Afterwards, it performs again the canny edge detector on the same edge picture to reduce noise and for a better smoothing it uses a 0.9 time the default threshold.





CODES

Code One

```
%This code takes the image, converts to a gray scale and then find its
%edges to draw a box around its shape.

clc
close all
clear all

%uploading picture
im=imread('Auto10.jpg');
imSm = imresize(im, .20);
figure,imshow(imSm);

%converting to a gray scale
imSmGray = rgb2gray(imSm);
imSmGray = im2double(imSmGray);

%displaying gray scale pictures
figure, imshow(imSmGray)
figure, imshow(im2bw(imSmGray,graythresh(imSmGray)))

%using maximum and minimum values
maxi=max(imSmGray(:));
mini=min(imSmGray(:));
figure, imhist(imSmGray)

%displaying image in red, green, blue pixels level.
figure, subplot(2,3,1),imshow(imSm(:,:,1)),title('Red')
subplot(2,3,2),imshow(imSm(:,:,2)),title('Green')
subplot(2,3,3),imshow(imSm(:,:,3)),title('Blue')

%tresholoding previous images.
subplot(2,3,4),imshow(im2bw(imSm(:,:,1),graythresh(imSm(:,:,1))))
title('Red Threshold')
subplot(2,3,5),imshow(im2bw(imSm(:,:,2),graythresh(imSm(:,:,2))))
title('Green Threshold')
subplot(2,3,6),imshow(im2bw(imSm(:,:,3),graythresh(imSm(:,:,3))))
title('Blue Threshold')

%using hsb function.
imH = rgb2hsv(imSm);
figure, imshow(imH)
```

```
%displaying result
figure, subplot(1,3,1),imshow(imH(:,:,1)),title('Hue')
subplot(1,3,2),imshow(imH(:,:,2)),title('Saturation')
subplot(1,3,3),imshow(imH(:,:,3)),title('Value')

%displaying histogram of Hue
figure,imhist(imH(:,:,1));
level = graythresh(imH(:,:,3)); % using best edged picture to draw box
imshow(imH(:,:,1)>level);

% BW is the binary image obtained by thresholding IMG
stats = regionprops((imH(:,:,1)>level),'BoundingBox');
figure
hold on
for object = 1 : length(stats)

   bb = stats(object).BoundingBox;
   rectangle('Position',bb,'EdgeColor','g','LineWidth',3)

end
hold off

imshow(imSm);


%drawing box
for object = 1 : length(stats)
   bb(object,:) = stats(object,:).BoundingBox;
   if(bb(object,1) > 5) && (bb(object,1) < 35)
     if(bb(object,2) > 5) && (bb(object,2) < 35)
   rectangle('Position',[bb(object,1) bb(object,1)  50 25],'EdgeColor','g','LineWidth',2)
   break
     end
   end
```

Code Two

```
clc
clear all
close all

%uploading picture
im=imread('Auto10.jpg');
imSm = imresize(im, 1);
figure,imshow(imSm);

%converting to a gray scale
imSmGray = rgb2gray(imSm);
imSmGray = im2double(imSmGray);

%displaying gray scale pictures
figure, imshow(imSmGray)
figure, imshow(im2bw(imSmGray,graythresh(imSmGray)))

%using maximum and minimum values
maxi=max(imSmGray(:))
mini=min(imSmGray(:))
figure, imhist(imSmGray)

%displaying image in red, green, blue pixels level.
figure, subplot(2,3,1),imshow(imSm(:,:,1)),title('Red')
subplot(2,3,2),imshow(imSm(:,:,2)),title('Green')
subplot(2,3,3),imshow(imSm(:,:,3)),title('Blue')

%tresholding previous images.
subplot(2,3,4),imshow(im2bw(imSm(:,:,1),graythresh(imSm(:,:,1))))
title('Red Threshold')
subplot(2,3,5),imshow(im2bw(imSm(:,:,2),graythresh(imSm(:,:,2))))
title('Green Threshold')
subplot(2,3,6),imshow(im2bw(imSm(:,:,3),graythresh(imSm(:,:,3))))
title('Blue Threshold')

%using hsb function.
imH = rgb2hsv(imSm);
figure, imshow(imH)

%displaying result
figure, subplot(1,3,1),imshow(imH(:,:,1)),title('Hue')
subplot(1,3,2),imshow(imH(:,:,2)),title('Saturation')
subplot(1,3,3),imshow(imH(:,:,3)),title('Value')

%displaying histogram of Hue
figure,imhist(imH(:,:,1));
level = graythresh(imH(:,:,3)); % using best edged picture to draw box
imshow(imH(:,:,1)>level);

% BW is the binary image obtained by thresholding IMG
stats = regionprops((imH(:,:,1)>level),'BoundingBox');
```

```
figure
hold on
for object = 1 : length(stats)

   bb = stats(object).BoundingBox;
   rectangle('Position',bb,'EdgeColor','g','LineWidth',3)

end
hold off

imshow(imSm);
%drawing box
for object = 1 : length(stats)
   bb(object,:) = stats(object,:).BoundingBox;
   if(bb(object,1) > 55) & (bb(object,1) < 105)
     if(bb(object,2) > 85) & (bb(object,2) < 135)
   rectangle('Position',[bb(object,1)-80 bb(object,1)  270 100],'EdgeColor','g','LineWidth',2)
   break
     end
   end

end
%hold off
I=im
G= rgb2gray(I);

%resizing image
E = imresize(G, .25);

%finding edges of the car and thresholding to find circles( the tires)
E=edge(G,'Canny',[.2 .75], .1);
BW = edge(E,'Canny', .92);
%figure, imshow(BW)

[H, theta, rho] = hough(E);
%figure, imshow(H, []);
peaks = houghpeaks(H, 50, 'Threshold', 30);

% BW is the binary image obtained by thresholding IMG
stats = regionprops(BW,'BoundingBox');

for object = 1 : length(stats)
   bb1 = stats(object,:).BoundingBox;
   %if( bb(object,2)>100)

%    bb(object,1) = stats(object,:).BoundingBox;
   rectangle('curvature',[1 1],'Position',bb1,'EdgeColor','r','LineWidth',2)
   %count=count+1
   %nd

end
hold off
```

CONCLUSION

A threshold invariant algorithm has been proposed that uses a modification of the Canny edge detector. The algorithm has been tested on a group of 10 images using Matlab and found to have reduced sensitivity to changes in image intensity compared with the Canny edge detector. One on the most challenging tasks in this project was the car extraction in images. A solution to this problem was to provide an algorithm that can be used to find any shape within an image, and then classify it accordingly to parameters needed to describe the shapes. One of the techniques used to achieve this is the Hough Transform. The Hough Transform often referred to as Circular Hough Transform was also used to identify circular objects in low-contrast noisy images. This method is highly dependent on converting gray-scale images to binary images using edge detection techniques such as Canny. The goal of this technique was to find irregular instances of objects within a pre-defined set of shapes by a voting procedure.

This project showed that images retrieved by using the global color histogram may not be semantically related even though they share similar color distribution in some results. In general, histogram-based retrievals in HSV color space showed better performance than in RGB color space since it showed higher precision values for the same recall values. In conclusion, the Histogram Intersection-based image retrieval in HSV color space is most desirable among two retrieval methods mentioned in considering computation to find car shapes. For future work, it is possible to use the histogram intersection distance measure with other features such as shape and texture to improve the overall image retrieval performance. Finding good parameters for quadratic distance measures could be another subject to improve histogram-based search algorithms. However, even in that case, it would be difficult to reduce the computation time for quadratic measurements significantly.

QUESTION

What the mathematical expression for RGB to HSB conversion?

Answer:

$$H = \cos^{-1}\left\{\frac{\frac{1}{2}[(R-G)+(R-B)]}{\sqrt{(R-G)^2+(R-B)(G-B)}}\right\}$$

$$S = 1 - \frac{3}{R+G+B}[\min(R,G,B)]$$

$$V = \frac{1}{3}(R+G+B)$$